

TD - Complexité algorithmique

September 21, 2026

1 Énoncé

Exercice 1

1. Écrire un programme permettant de calculer la somme des n premiers nombres naturels :
 - En utilisant la méthode additionnelle (boucle).
 - En utilisant la formule : $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.
2. Calculer et comparer la complexité des deux programmes.

Exercice 2

1. Écrire une fonction qui renvoie le maximum d'une liste de nombres.
2. Calculer la complexité de l'algorithme pour les trois cas : meilleur, moyen et pire.

Exercice 3

1. Écrire une fonction qui teste si une chaîne de caractères est un palindrome.
2. Calculer sa complexité (meilleur et pire cas), en fonction de n , la longueur de la chaîne.

Exercice 4

Soit la fonction suivante :

```
[1]: def compte_divisions(n, k):  
    i=1  
    compteur = 0  
    while i < n:  
        i=i*k  
        compteur += 1  
    return compteur
```

1. Donner sa complexité en fonction de n et k (avec $k \geq 2$).
2. Ce résultat dépend-il vraiment de k en notation $O(\dots)$?

Exercice 5

1. Réécrire le programme de tri d'une liste par insertion.
2. Calculer la complexité de l'algorithme pour les deux cas, meilleur et pire.
3. Est-ce le même résultat pour le tri par sélection?

2 Correction

Exercice 1

1.

```
[2]: def somme_boucle(n):  
    s=0  
    for i in range(1, n+1):  
        s += i  
    return s  
  
def somme_formule(n):  
    return n*(n+1)//2
```

2.

- Méthode par boucle : la boucle effectue exactement n itérations, chacune coûtant $O(1)$ (une lecture, une addition et une affectation) : $T(n) = O(n)$.
- Méthode par formule : quelle que soit la valeur de n , on effectue toujours le même nombre fixe d'opérations élémentaires (une addition, une multiplication, une division entière) : $T(n) = O(1)$.

La méthode par formule est asymptotiquement bien meilleure — constante contre linéaire.

Exercice 2

1.

```
[3]: def maximum(liste):  
    maxi = liste[0]  
    for x in liste[1:]:  
        if x > maxi:  
            maxi = x  
    return maxi
```

2.

Trouver le maximum oblige à examiner chaque élément : on ne peut jamais être sûr qu'un élément est le maximum sans l'avoir comparé à tous les autres. D'où $T_{meilleur}(n) = T_{moyen}(n) = T_{pire}(n) = \Theta(n)$.

Exercice 3

1.

```
[4]: def est_palindrome(mot):  
    n = len(mot)  
    for i in range(n // 2):  
        if mot[i] != mot[n - 1 - i]:  
            return False  
    return True
```

2.

- Meilleur cas : Le premier et le dernier caractère diffèrent déjà — sortie immédiate, donc $T(n) = O(1)$.
- Pire cas : La chaîne est effectivement un palindrome — il faut comparer toutes les $\frac{n}{2}$ paires, donc $T(n) = O(n)$.

Exercice 4

1. Après d itérations, $i = k^d$. La boucle s'arrête quand $k^d \geq n$, donc $d = \log_k(n)$: $T(n) = O(\log_k(n))$.
2. Par changement de base, $\log_k(n) = \frac{\log(n)}{\log(k)}$. Le facteur $\frac{1}{\log(k)}$ est une constante (elle ne dépend pas de n), donc elle disparaît dans la notation O , quel que soit $k \geq 2$, on a toujours $T(n) = O(\log n)$.

Exercice 5

1.

```
[5]: def tri_par_insertion(liste):
    for i in range(1, len(liste)):
        x = liste[i]
        p = i-1
        while x < liste[p] and p >= 0:
            liste[p+1] = liste[p]
            p -= 1
        liste[p+1] = x
        print(liste)
    return liste

liste = [5,1,1,7,2,3,2,3]
tri_par_insertion(liste)
```

```
[1, 5, 1, 7, 2, 3, 2, 3]
[1, 1, 5, 7, 2, 3, 2, 3]
[1, 1, 5, 7, 2, 3, 2, 3]
[1, 1, 2, 5, 7, 3, 2, 3]
[1, 1, 2, 3, 5, 7, 2, 3]
[1, 1, 2, 2, 3, 5, 7, 3]
[1, 1, 2, 2, 3, 3, 5, 7]
```

```
[5]: [1, 1, 2, 2, 3, 3, 5, 7]
```

2. Complexité dans les deux cas :

- Meilleur cas : la liste est déjà triée. Pour chaque i , la condition $liste[p] > x$ est fausse dès le premier test, la boucle while ne s'exécute jamais. Coût constant par itération de la boucle for : $T(n) = O(n)$.
- Pire cas : la liste est triée en ordre décroissant. Chaque élément doit être décalé jusqu'au tout début : à l'étape i , il faut i décalages. Le total est $1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2}$:

$$T(n) = O(n^2).$$

3. Dans le cas du tri par sélection, à chaque étape, on cherche le minimum du reste du tableau — cette recherche parcourt systématiquement tous les éléments restants, quel que soit leur ordre : le meilleur cas est aussi $\Theta(n^2)$, exactement comme dans l'exercice 2 sur la recherche du maximum.